```
    iounmap(p->screen_base);
    framebuffer_release(p);
}
```

The above code is an example of a very simple probe function. Now, let's talk a little about what we have used in our probe function.

framebuffer_alloc: This creates a new framebuffer information structure. It also reserves the size for the driver's private data (*info->par*). You might have guessed by now that *ourfb_par* is our private data. This function will return a pointer to struct *fb_info* or it returns NULL if it fails.

ourfb_par: As already said, this is the private data of our driver. It is not a required component but it is recommended that you have it in your driver. At any point of time, this structure will have information about the hardware state of the graphics card.

framebuffer_virtual_memory: This is not defined in our code, but it is actually the virtual memory address for the framebuffer. We get this address after remapping the resource address. The following code will show you how we get this address:

```
    unsigned long addr, size;
    addr = pci_resource_start(dev, 0);
    size = pci_resource_len(dev, 0);
    if (addr == 0)
      return -ENODEV;
    framebuffer_virtual_memory  = ioremap(addr, 0x800000);
```

ourfb_ops: This structure will define the operations that we will allow on our framebuffer. We can have many operations here which are similar to any character device driver, like open, read, write, release, ioctl, etc. But the operations that we have to provide are *fb_fillrect*, *fb_copyarea* and *fb_imageblit*. Regarding the other operations, even if we do not mention them in our ops structure, they will be taken care of automatically by the framebuffer layer by default. If we need any customised functions for our driver, then we have to provide for them also.

Fortunately, the framebuffer layer has three functions: *cfb_fillrect*, *cfb_imageblit* and *cfb_copyarea*. We can use these functions in our driver instead of writing our own *fb_fillrect*, *fb_copyarea* and *fb_imageblit* functions. But if you are writing a driver for hardware that supports hardware acceleration, then you have to write your own functions.

```
static struct fb_ops ourfb_ops = {
    .owner        = THIS_MODULE,
    .fb_fillrect  = cfb_fillrect,
    .fb_imageblit = cfb_imageblit,
    .fb_copyarea  = cfb_copyarea,
};
```

ourfb_fix: This structure will contain fixed information about our hardware. The following is the structure as defined in the header file:

```
struct fb_fix_screeninfo {
    char id[16];    /* identification string */
    unsigned long smem_start;       /* Start of frame buffer
mem (physical address) */
__u32 smem_len;   /* Length of frame buffer mem */
__u32 type;                 /* FB_TYPE_*                */
__u32 type_aux;      /* Interleave for interleaved Planes */
__u32 visual;           /*   FB_VISUAL_*            */
__u16 xpanstep;         /* zero if no hardware panning  */
__u16 ypanstep;       /* zero if no hardware panning  */
__u16 ywrapstep;         /* zero if no hardware ywrap    */
__u32 line_length;       /* length of a line in bytes    */
unsigned long mmio_start;      /* Start of Memory Mapped I/O
(physical address) */
__u32 mmio_len;         /* Length of Memory Mapped I/O  */
__u32 accel;              /* Indicate to driver which
                specific chip/card we have  */
__u16 capabilities;       /* FB_CAP_*                 */
__u16 reserved[2];   /* Reserved for future compatibility */
};
```

flags: This will indicate the type of hardware acceleration our driver is capable of.

fb_alloc_cmap: This allocates memory for the colour map, which our driver is going to use.

register_framebuffer: This is the most important function that will actually register our framebuffer with the framebuffer layer. This function is responsible for creating the device nodes */dev/fb**.

Note: In the probe function, at any stage, if the initialisation fails, we have to undo whatever we have done before that, and then need to return an appropriate error value.

In the *ourfb_remove* function, we are unwinding what we have done in the probe function in a reverse manner.

We have just looked at how to create a very simple framebuffer device, though we have not touched upon any hardware related topics here. But when we work with an actual framebuffer driver that is responsible for video hardware, we need to give all the hardware initialisation routines in the probe function. If you want to see the code of any framebuffer device driver, you can find it in the *drivers/video/fbdev/* folder of the Linux source tree.

Another complicated scenario: Some graphical hardware will have support for two monitors, where each display can have its own unique data. In this case, each display should have its own separate framebuffer device, which means separate struct *fb_info*. Some hardware will have the double-buffering capability. END

By: Sudip Mukherjee

The author is the maintainer of the Silicon Motion SM712 framebuffer driver in the mainline kernel. He works as an embedded Linux developer at Vector India Pvt Ltd, a renowned embedded training institute. He can be reached at *sudip@vectorindia.org*